Manual de implementación del sistema Tu Evento

Jhampier Santos Ortiz y Cristofer David Lozano Contreras

Servicio Nacional de Aprendizaje (SENA)

Sede de la Industria y los Servicios, Neiva, Huila

Tecnólogo en Análisis y Desarrollo de Software

Instructor: Carlos Julio Cadena

9 de octubre de 2026

Contenido

Introducción . . . 3
Arquitectura del sistema . . . 3
Visión general . . . 3
Requisitos . . . 5
Hardware . . . 5
Software . . . 6
Puertos . . . 6
Obtención del código y variables de entorno . . . 7
Despliegue con Docker Compose . . . 9
Servicios y orden de arranque . . . 9
Migraciones y administrador inicial . . . 11
Detención y reinicio . . . 11
Despliegue en Kubernetes . . . 12
Integración continua con Jenkins . . . 13
Desarrollo local . . . 15
Backend . . . 15
Frontend web . . . 15
Aplicación móvil . . . 15
Compilación de la aplicación móvil . . . 16
Verificación del sistema . . . 16
Seguridad . . . 17
Solución de problemas . . . 18
Limitaciones conocidas . . . 18
Referencias . . . 20

Manual de implementación del sistema Tu Evento

Introducción

El presente manual describe el procedimiento para instalar, configurar, desplegar y verificar el sistema Tu Evento, una plataforma de gestión de eventos dirigida a usuarios finales, organizadores y administradores. Está orientado a quienes deban poner el sistema en funcionamiento, ya sea en un equipo de desarrollo mediante Docker Compose o en un clúster de Kubernetes.

La información fue contrastada con los archivos de configuración del repositorio del proyecto (Santos Ortiz y Lozano Contreras, 2026), en particular el archivo `docker-compose.yml`, los manifiestos de la carpeta `k8s/`, el `Jenkinsfile` y los archivos de configuración de Spring Boot. Por motivos de seguridad, el documento no contiene valores secretos: cada credencial se identifica únicamente por el nombre de su variable de entorno.

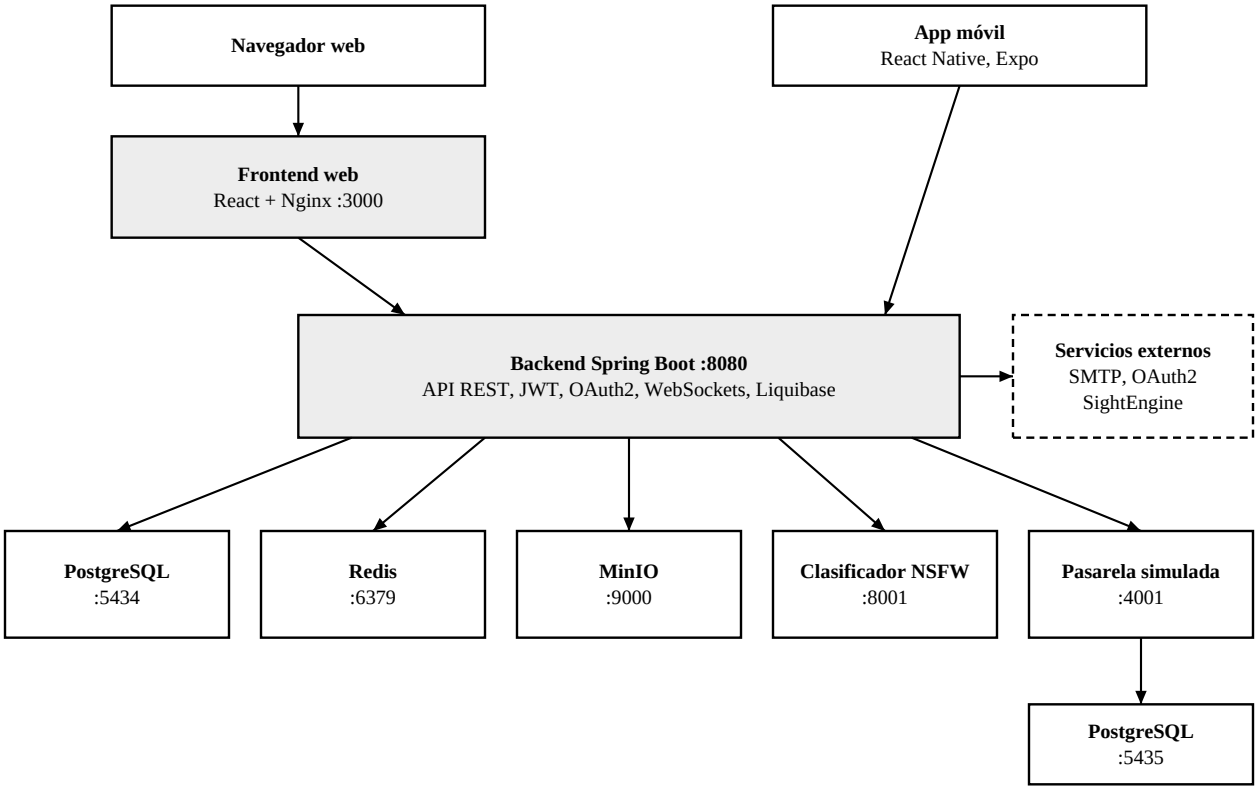
Arquitectura del sistema

Visión general

Tu Evento sigue una arquitectura cliente-servidor. El backend, desarrollado con Spring Boot (Spring, s. f.), es una aplicación monolítica que expone una API REST y se apoya en servicios auxiliares: PostgreSQL para la persistencia de datos (PostgreSQL Global Development Group, s. f.), Redis como caché (Redis Ltd., s. f.) y MinIO para el almacenamiento de objetos (MinIO, Inc., s. f.). La Figura 1 presenta la relación entre los componentes y los puertos publicados en el entorno de desarrollo, y la Tabla 1 resume la función de cada uno.

Figura 1

Arquitectura general del sistema Tu Evento



Nota. Diagrama de elaboración propia a partir del archivo docker-compose.yml del proyecto. Las flechas indican el sentido de las solicitudes; los números precedidos de dos puntos corresponden a los puertos publicados en el equipo anfitrión. Los servicios externos (correo SMTP, proveedores OAuth2 y SightEngine) se representan con borde discontinuo.

Tabla 1
Componentes del sistema

Componente	Tecnología	Función
Backend	Spring Boot 3.5, Java 21	API REST, lógica de negocio, seguridad (JWT y OAuth2), WebSockets y migraciones con Liquibase.
Frontend web	React 18, Vite 5	Interfaz web; en contenedor se sirve con Nginx.
Frontend móvil	React Native 0.81, Expo SDK 54	Aplicación para Android e iOS.
Base de datos	PostgreSQL 17	Persistencia de datos del backend.
Caché	Redis 7	Caché y datos de sesión.
Almacenamiento de objetos	MinIO	Imágenes de perfil, medios de eventos y documentos de organizadores.
Moderación de imágenes	Servicio NSFW (Python 3.11) y SightEngine	Clasificación del contenido antes de almacenarlo.
Pasarela de pagos	fake-payment-gateway (Java 21)	Pasarela simulada con su propia base de datos PostgreSQL.
Integración continua	Jenkins y SonarQube	Construcción, empaquetado y análisis de calidad.

Nota. Elaboración propia a partir de los archivos pom.xml, package.json y docker-compose.yml del proyecto.

Requisitos

Hardware

El stack completo ejecuta once contenedores, entre ellos un modelo de clasificación de imágenes, Jenkins y SonarQube. La Tabla 2 indica los recursos recomendados para el equipo anfitrión.

Tabla 2
Recursos recomendados del equipo anfitrión

Recurso	Mínimo	Recomendado
Memoria RAM	8 GB (sin Jenkins ni SonarQube)	16 GB (stack completo)
Procesador	4 núcleos	6 núcleos o más
Almacenamiento libre	20 GB	40 GB
Sistema operativo	Windows 10/11 con WSL2, Ubuntu 20.04 o superior, macOS	Ubuntu 22.04 LTS
Red	Acceso a Internet para descargar imágenes y dependencias	Igual

Nota. Los valores son recomendaciones de operación del equipo de desarrollo y deben ajustarse a la carga real.

Software

Las herramientas necesarias dependen del escenario de trabajo, como se detalla en la Tabla 3.

Tabla 3
Herramientas de software según el escenario

Escenario	Herramientas necesarias
Despliegue con Docker Compose	Docker Engine, Docker Compose v2 y Git.
Desarrollo del backend	JDK 21 y Maven (el proyecto incluye Maven Wrapper, `mvnw`).
Desarrollo del frontend web	Node.js 20 y npm.
Desarrollo del frontend móvil	Node.js 20, npm y EAS CLI 22.2.0 o superior; Android Studio o Xcode para emuladores (opcional).
Despliegue en Kubernetes	kubectl y un clúster con controlador Ingress NGINX, cert-manager y metrics-server.

Puertos

Los puertos publicados en el equipo anfitrión por cada servicio se listan en la Tabla 4. Deben estar libres antes de iniciar el sistema.

Tabla 4
Puertos publicados por los servicios

Puerto	Servicio
3000	Frontend web (Nginx en Docker)
5173	Frontend web en modo desarrollo (Vite)
8080	Backend (API)
5434	PostgreSQL del backend (puerto 5432 dentro del contenedor)
5435	PostgreSQL de la pasarela simulada (puerto 5432 dentro del contenedor)
6379	Redis
9000 y 9001	MinIO (API y consola web)
8001	Servicio de clasificación NSFW
4001	Pasarela de pagos simulada
8082 y 50000	Jenkins (interfaz y agentes)
9002	SonarQube

Nota. Fuente: docker-compose.yml del proyecto.

Obtención del código y variables de entorno

Todos los servicios se orquestan desde el archivo `docker-compose.yml`, ubicado en la raíz del repositorio, y las variables se leen desde un archivo `.env` en esa misma carpeta, como indica la documentación de Docker Compose (Docker Inc., s. f.). El Código 1 muestra cómo obtener el código y crear el archivo de variables.

Código 1

Obtención del código y creación del archivo de variables

```
git clone <URL_DEL_REPOSITORIO> TuEvento
cd TuEvento
cp .env.example .env
```

Después de crear el archivo, se deben completar las variables de la Tabla 5 con valores propios; no se deben reutilizar credenciales de ejemplos ni de documentos anteriores.

Tabla 5

Variables de entorno definidas en el archivo .env.example

Variable	Descripción
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	Base de datos, usuario y contraseña de PostgreSQL (compartidos con la pasarela simulada).
MAIL_USERNAME, MAIL_PASSWORD	Cuenta SMTP de Gmail y su contraseña de aplicación.
JWT_SECRET	Clave para firmar tokens JWT (al menos 256 bits).
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Credenciales OAuth2 de Google.
VITE_GOOGLE_CLIENT_ID	Client ID de Google usado por el frontend web.
FACEBOOK_CLIENT_ID, FACEBOOK_CLIENT_SECRET	Credenciales OAuth2 de Facebook.
ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_ALIAS	Datos del administrador inicial creado en el primer arranque.
MINIO_ROOT_USER, MINIO_ROOT_PASSWORD	Usuario y contraseña raíz de MinIO (también actúan como clave de acceso).
REDIS_HOST, REDIS_PORT	Ubicación de Redis.
SPRING_PROFILES_ACTIVE	Perfil de Spring: dev, k8s o prod.
NSFW_SERVICE_URL, NSFW_THRESHOLD	URL interna y umbral del clasificador NSFW.
SIGHTENGINE_API_USER, SIGHTENGINE_API_SECRET, SIGHTENGINE_MODELS, SIGHTENGINE_THRESHOLD	Moderación adicional con SightEngine.
NOTIFICATION_EMAIL_ENABLED, NOTIFICATION_MAIL_HOST, NOTIFICATION_MAIL_PORT, NOTIFICATION_MAIL_USERNAME, NOTIFICATION_MAIL_PASSWORD	Notificaciones por correo electrónico.

Nota. Fuente: archivo .env.example del proyecto. No se incluyen valores.

Además, el archivo `docker-compose.yml` y los perfiles de Spring utilizan las variables de la Tabla 6, que no figuran en `.env.example` y deben agregarse al archivo `.env` cuando corresponda. La ausencia de `WEBHOOK_SECRET` impide el arranque correcto de la pasarela simulada.

Tabla 6

Variables utilizadas que no están en .env.example

Variable	Descripción
WEBHOOK_SECRET	Secreto para validar webhooks de la pasarela simulada; obligatorio para el servicio fake-payment-gateway.
MINIO_PUBLIC_URL	URL de MinIO accesible desde el navegador, usada en URL prefirmadas.
PAYMENT_GATEWAY	Selector de pasarela; el valor `fake` activa la pasarela simulada.
FAKE_GATEWAY_URL	URL de la pasarela simulada vista desde el backend.
SIGHTENGINE_NUDITY_THRESHOLD	Umbral específico de detección de desnudez.
APP_CORS_ALLOWED_ORIGINS	Orígenes permitidos por CORS (incluir http://localhost:3000 y http://localhost:5173).
APP_STORAGE_ENDPOINT	Endpoint interno de MinIO para cargar y eliminar archivos.

Nota. Fuente: docker-compose.yml y application-dev.yml del proyecto.

Despliegue con Docker Compose

Este es el método recomendado para entornos de desarrollo y pruebas. Todos los comandos del Código 2 se ejecutan desde la raíz del repositorio, que es donde se encuentra el archivo `docker-compose.yml`; no desde la carpeta del backend.

Código 2

Inicio del sistema completo con Docker Compose

```
docker compose up --build -d
docker compose ps
docker compose logs -f backend
```

Servicios y orden de arranque

Docker Compose permite declarar dependencias entre servicios para controlar el orden de inicio (Docker Inc., s. f.). La Tabla 7 describe los once servicios del proyecto y la Figura 2 resume las dependencias que condicionan el arranque del backend y del frontend.

Tabla 7

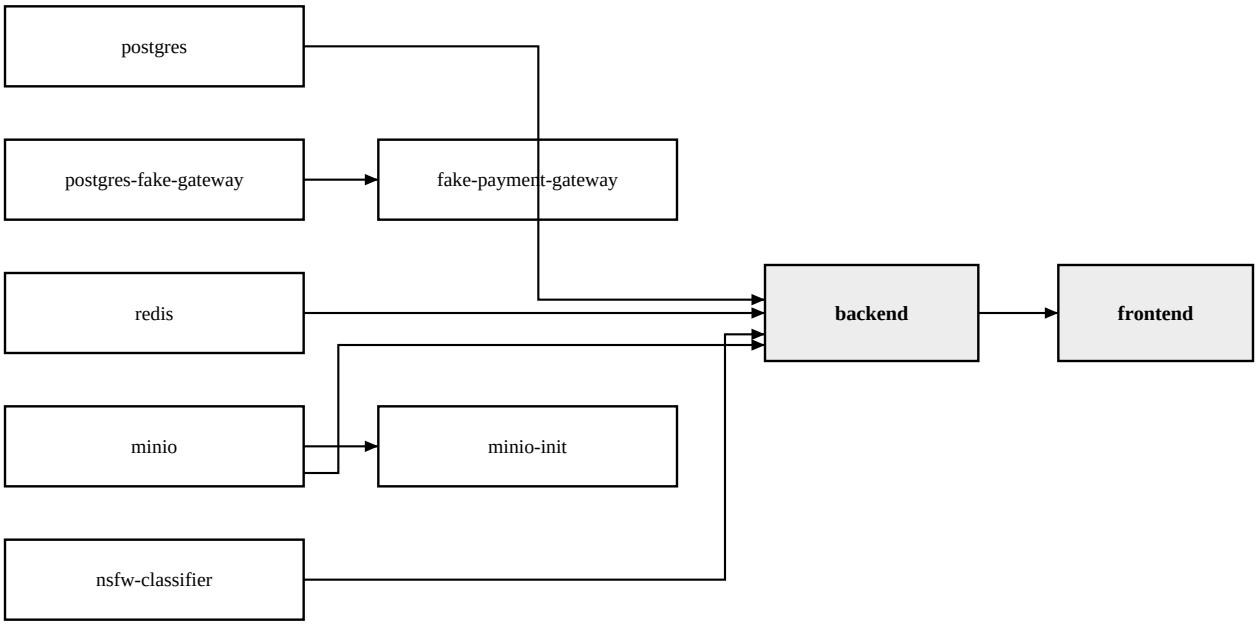
Servicios definidos en docker-compose.yml

Servicio	Imagen o construcción	Puertos	Depende de
postgres	postgres:17-alpine	5434:5432	-
postgres-fake-gateway	postgres:17-alpine	5435:5432	-
redis	redis:7-alpine	6379	-
minio	quay.io/minio/minio	9000, 9001	-
minio-init	quay.io/minio/minio	-	minio (saludable)
nsfw-classifier	Construcción local (Python 3.11)	8001:8000	-
fake-payment-gateway	Construcción local (Java 21)	4001	postgres-fake-gateway
backend	Construcción local (Java 21)	8080	postgres, minio, redis, nsfw-classifier
frontend	Construcción local (Node 20 y Nginx)	3000:80	backend
jenkins	jenkins/jenkins:lts-jdk17	8082, 50000	-
sonarqube	sonarqube:lts-community	9002:9000	postgres

Nota. Fuente: docker-compose.yml del proyecto.

Figura 2

Dependencias de arranque entre servicios



Nota. Diagrama de elaboración propia a partir de las cláusulas depends_on del archivo docker-compose.yml. Cada flecha va del servicio que debe iniciar primero al que depende de él. Jenkins no tiene dependencias y SonarQube depende de postgres; ambos se omiten para simplificar.

El contenedor `minio-init` crea el bucket `tuevento`. Los prefijos `profile_picture/` y `event_media/` son públicos, mientras que `organizer_document/` es privado.

Migraciones y administrador inicial

No se requiere ningún paso manual sobre la base de datos. Al iniciar, el backend ejecuta automáticamente las migraciones de Liquibase (Liquibase, s. f.), más de noventa archivos ubicados en `db/changelog/changes/`, que incluyen los datos de departamentos y ciudades. Después, el componente `DataInitializer` crea el usuario administrador a partir de `ADMIN_EMAIL`, `ADMIN_PASSWORD` y `ADMIN_ALIAS`, solo si no existe previamente.

Detención y reinicio

El Código 3 muestra los comandos para detener, reiniciar o eliminar por completo el entorno. La opción `-v` borra los volúmenes y, por lo tanto, los datos almacenados.

Código 3

Detención y reinicio del entorno

```
docker compose down            # detiene y elimina contenedores
docker compose down -v         # ademas elimina volumenes (borra datos)
docker compose up -d           # inicia nuevamente sin reconstruir
```

Despliegue en Kubernetes

Los manifiestos se encuentran en la carpeta `k8s/`. Se requiere un clúster activo con kubectl configurado, un controlador Ingress NGINX con cert-manager (emisor `letsencrypt-prod`) y metrics-server para el autoescalado. El Ingress expone los servicios mediante dominios y TLS (The Kubernetes Authors, s. f.-b) y el HorizontalPodAutoscaler ajusta el número de réplicas según el consumo de recursos (The Kubernetes Authors, s. f.-a). La Tabla 8 resume los recursos definidos.

Tabla 8

Manifiestos de Kubernetes del proyecto

Manifiesto	Recurso	Detalle
secrets.yaml	Secret tu-evento-secrets	Credenciales de base de datos, MinIO, JWT, OAuth, administrador y correo.
configmap.yaml	ConfigMap tu-evento-config	Perfil prod, redis-service:6379 y postgres-service:5432.
deployment.yaml y service.yaml	Backend	Puerto 8080; servicio 80 a 8080, NodePort 30080.
frontend-deployment.yaml y frontend-service.yaml	Frontend	Puerto 80; NodePort 30081.
ingress.yaml	Ingress	api.tuevento.com y app.tuevento.com con TLS.
hpa.yaml	Autoescalado	Entre 2 y 10 réplicas del backend; CPU 70 % y memoria 80 %.

Nota. Fuente: carpeta k8s/ del proyecto.

Los manifiestos deben aplicarse en el orden del Código 4.

Código 4

Aplicación ordenada de los manifiestos

```
kubectl apply -f k8s/secrets.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/frontend-deployment.yaml
kubectl apply -f k8s/frontend-service.yaml
kubectl apply -f k8s/ingress.yaml
kubectl apply -f k8s/hpa.yaml
kubectl get pods
```

Deben tenerse en cuenta tres consideraciones. Primero, los manifiestos no incluyen PostgreSQL, Redis ni MinIO, por lo que estos servicios deben existir en el clúster con los nombres postgres-service, redis-service y minio-service. Segundo, el perfil k8s tiene el almacenamiento deshabilitado (`app.storage.enabled=false`), de modo que la carga de archivos no estará disponible hasta configurarlo. Tercero, antes de aplicar `secrets.yaml` deben reemplazarse los valores de ejemplo por valores propios codificados en base64, y ese archivo no debe versionarse con datos reales.

Integración continua con Jenkins

Jenkins se ejecuta como contenedor y está disponible en el puerto 8082. El `Jenkinsfile` de la raíz define el proceso como código (Jenkins, s. f.), con las etapas de la Tabla 9 y el flujo de la Figura 3.

Tabla 9

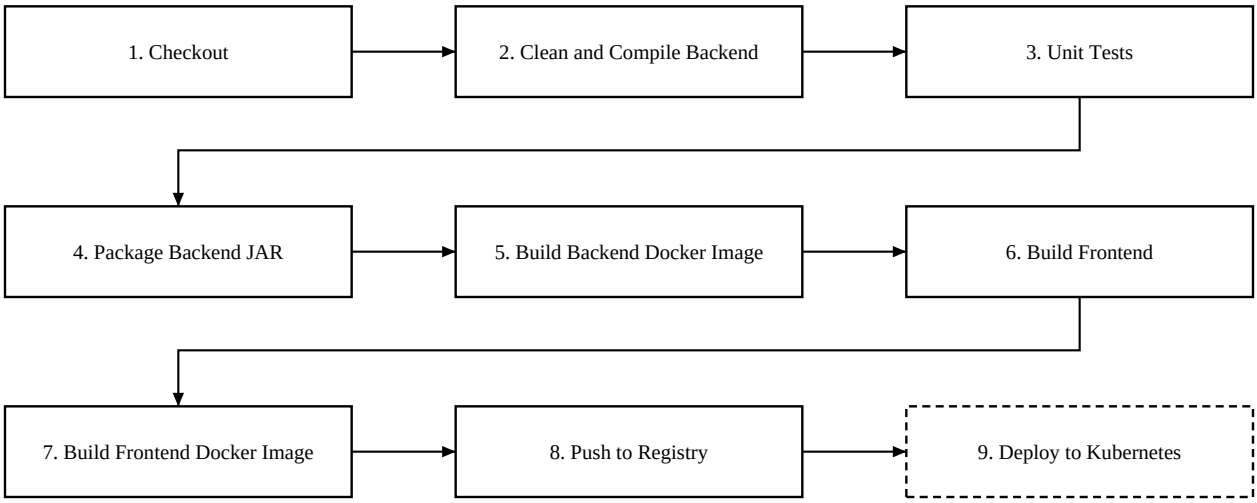
Etapas del Jenkinsfile

Etapa	Acción
Checkout	Clona el repositorio.
Clean and Compile Backend	./mvnw clean compile en TuEventoBackend/tu-evento.
Unit Tests	./mvnw test con la opción -Dmaven.test.skip=true; por tanto, las pruebas actualmente no se ejecutan.
Package Backend JAR	./mvnw clean package -DskipTests.
Build Backend Docker Image	docker build de la imagen del backend.
Build Frontend	npm ci y npm run build en TuEventoFrontend/web.
Build Frontend Docker Image	docker build de la imagen del frontend.
Push to Registry	Publica las imágenes en Docker Hub con las etiquetas del número de build y latest.
Deploy to Kubernetes	Etapa informativa: solo imprime mensajes y no aplica manifiestos.

Nota. Fuente: Jenkinsfile del proyecto.

Figura 3

Flujo de etapas del pipeline de Jenkins



Nota. Diagrama de elaboración propia a partir del Jenkinsfile. Las etapas se ejecutan de izquierda a derecha y de arriba hacia abajo. La última etapa, con borde discontinuo, es solo informativa.

En consecuencia, el despliegue en Kubernetes debe realizarse manualmente según se indicó en la sección Despliegue en Kubernetes, hasta que la etapa final del pipeline se implemente.

Desarrollo local

Backend

Primero se inicia la infraestructura con Docker Compose y luego se ejecuta el backend con el perfil `dev` y las variables de entorno exportadas, como se muestra en el Código 5. El archivo generado por el empaquetado es `tu-evento-0.0.1-SNAPSHOT.jar`.

Código 5

Ejecución del backend en modo desarrollo

```
docker compose up -d postgres redis minio minio-init nsfw-classifier
cd TuEventoBackend/tu-evento
./mvnw clean package -DskipTests
./mvnw spring-boot:run
# o bien:
java -jar target/tu-evento-0.0.1-SNAPSHOT.jar
```

Frontend web

El frontend usa Vite (Vite, s. f.). En modo desarrollo se atiende normalmente en el puerto 5173, mientras que el puerto 3000 corresponde al contenedor con Nginx. La variable `VITE_API_URL` define la dirección de la API y, en desarrollo, vale `http://localhost:8080/api`. El Código 6 resume los comandos.

Código 6

Ejecución y compilación del frontend web

```
cd TuEventoFrontend/web
npm install
npm run dev        # Vite, normalmente en http://localhost:5173
npm run build      # genera la carpeta dist
```

Aplicación móvil

La variable que apunta al backend es `EXPO_PUBLIC_API_URL`. Expo solo incluye en el paquete de la aplicación las variables cuyo nombre empieza por `EXPO_PUBLIC_` (Expo, s. f.-a). En un dispositivo físico o emulador no debe usarse `localhost`, sino la dirección IP del equipo que ejecuta el backend (en el emulador de Android, 10.0.2.2). Los comandos se muestran en el Código 7.

Código 7

Ejecución de la aplicación móvil en desarrollo

```
cd TuEventoFrontend/movil
npm install
npx expo start
npx expo run:android
npx expo run:ios            # solo en macOS
```

Compilación de la aplicación móvil

La compilación se realiza con EAS Build (Expo, s. f.-b), configurado en `eas.json`; requiere EAS CLI 22.2.0 o superior y una cuenta de Expo. El perfil `preview` genera un APK de distribución interna y el perfil `production` genera un paquete AAB para tienda. Los comandos `expo build:android` y `expo build:ios` están obsoletos y no funcionan con Expo SDK 54.

Código 8

Compilación de la aplicación móvil con EAS

```
npm install -g eas-cli
eas login
eas build --platform android --profile preview       # APK interno
eas build --platform android --profile production    # AAB para tienda
```

Verificación del sistema

Una vez iniciados los servicios, su estado se confirma con las direcciones de la Tabla 10. El backend expone los puntos de salud y métricas mediante Spring Boot Actuator (Spring, s. f.), que también utilizan las sondas de Kubernetes.

Tabla 10
Direcciones de verificación

Dirección	Descripción
http://localhost:8080/actuator/health	Estado del backend.
http://localhost:8080/actuator/metrics	Métricas de la aplicación.
http://localhost:8080/swagger-ui	Documentación interactiva de la API.
http://localhost:8080/api-docs	Especificación OpenAPI en JSON.
http://localhost:3000	Frontend web.
http://localhost:9000/minio/health/live	Estado de MinIO.
http://localhost:9001	Consola web de MinIO.
http://localhost:8001/health	Estado del clasificador NSFW.
http://localhost:4001	Pasarela de pagos simulada.
http://localhost:8082	Jenkins.
http://localhost:9002	SonarQube.

Nota. Fuente: docker-compose.yml y application-dev.yml del proyecto.

Como prueba funcional final, se debe abrir el frontend web e iniciar sesión con las credenciales definidas en `ADMIN_EMAIL` y `ADMIN_PASSWORD`.

Seguridad

- Las credenciales, claves y tokens no deben registrarse en este manual, en el repositorio ni en capturas de pantalla; solo se usa el archivo `.env`, que no debe versionarse, o los Secrets de Kubernetes.
- El `JWT_SECRET` debe ser aleatorio, de al menos 256 bits, y cada servicio debe tener contraseñas distintas.
- Toda credencial que haya aparecido en versiones anteriores de la documentación (contraseña SMTP, secretos OAuth, contraseña de base de datos) debe considerarse comprometida y rotarse.
- La variable `APP_CORS_ALLOWED_ORIGINS` debe contener únicamente los orígenes necesarios.

- En producción no deben exponerse a Internet los puertos de PostgreSQL, Redis, MinIO, Jenkins ni SonarQube; solo el frontend y la API, mediante el Ingress con TLS.
- Deben fijarse versiones concretas de las imágenes, por ejemplo la de MinIO, que usa la etiqueta `latest`, para lograr despliegues reproducibles.

Solución de problemas

La Tabla 11 reúne los síntomas más frecuentes y la acción recomendada.

Tabla 11

Problemas frecuentes y acciones recomendadas

Síntoma	Causa probable y acción
El backend no inicia	Verificar que PostgreSQL, Redis, MinIO y el clasificador NSFW estén saludables (docker compose ps), revisar docker compose logs backend y confirmar que el puerto 8080 esté libre.
Error de conexión a la base de datos	Revisar POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD en .env. Desde el equipo anfitrión, PostgreSQL se expone en el puerto 5434, no en el 5432.
Falla la pasarela de pagos	Verificar que WEBHOOK_SECRET esté definido en .env.
Las imágenes no cargan	Revisar MINIO_PUBLIC_URL y que minio-init haya creado el bucket tuevento.
Error de CORS en el navegador	Incluir el origen del frontend en APP_CORS_ALLOWED_ORIGINS (3000 en Docker, 5173 con Vite).
Falla el inicio de sesión con Google o Facebook	Comprobar las credenciales OAuth2 y que las URI de redirección coincidan con las registradas en cada proveedor.
La app móvil no se conecta al backend	Verificar que EXPO_PUBLIC_API_URL use la IP del equipo y no localhost; reiniciar con npx expo start -c.
El frontend web no compila	Usar Node.js 20; eliminar node_modules y ejecutar npm install.

Limitaciones conocidas

- La pasarela de pagos es simulada; la integración con Wompi no está implementada, aunque el enumerado de pasarelas y parte de la documentación pública la mencionan.
- El almacenamiento usa MinIO; la columna `s3_key` de la tabla de archivos conserva ese nombre por herencia y no existe dependencia de AWS S3.
- En Kubernetes el almacenamiento está deshabilitado y los servicios de datos no se incluyen en los manifiestos.

- La etapa de despliegue del Jenkinsfile no ejecuta acciones reales y la etapa de pruebas omite los tests.
- Los recursos de hardware indicados son recomendaciones y deben ajustarse a la carga real.

Referencias

Docker Inc. (s. f.). *Docker Compose overview*. Recuperado el 9 de octubre de 2026, de https://docs.docker.com/compose/

Expo. (s. f.-a). *Environment variables in Expo*. Recuperado el 9 de octubre de 2026, de https://docs.expo.dev/guides/environment-variables/

Expo. (s. f.-b). *Introduction to EAS Build*. Recuperado el 9 de octubre de 2026, de https://docs.expo.dev/build/introduction/

Jenkins. (s. f.). *Using a Jenkinsfile*. Recuperado el 9 de octubre de 2026, de https://www.jenkins.io/doc/book/pipeline/jenkinsfile/

Liquibase. (s. f.). *Liquibase documentation*. Recuperado el 9 de octubre de 2026, de https://docs.liquibase.com/

MinIO, Inc. (s. f.). *MinIO object storage documentation*. Recuperado el 9 de octubre de 2026, de https://min.io/docs/minio/linux/index.html

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 17 documentation*. Recuperado el 9 de octubre de 2026, de https://www.postgresql.org/docs/17/

Redis Ltd. (s. f.). *Redis documentation*. Recuperado el 9 de octubre de 2026, de https://redis.io/docs/

Santos Ortiz, J. y Lozano Contreras, C. D. (2026). *Tu Evento: Plataforma de gestión de eventos* [Software]. Repositorio del proyecto formativo.

Spring. (s. f.). *Spring Boot reference documentation*. Recuperado el 9 de octubre de 2026, de https://docs.spring.io/spring-boot/index.html

The Kubernetes Authors. (s. f.-a). *Horizontal Pod Autoscaling*. Recuperado el 9 de octubre de 2026, de https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

The Kubernetes Authors. (s. f.-b). *Ingress*. Recuperado el 9 de octubre de 2026, de https://kubernetes.io/docs/concepts/services-networking/ingress/

Vite. (s. f.). *Getting started*. Recuperado el 9 de octubre de 2026, de https://vite.dev/guide/